

INTRODUCTION AU LANGAGE



Moussa LO
M2ITIC/IFOAD 2014/2015
Université Gaston Berger de Saint-Louis

Bibliographie Java (1/2)

- **Le langage Java**, K.Arnlod, J. Gosling, Thomson, 1996.
- **Java La synthèse – Des concepts objet aux architectures Web**, G. Clavel, et al., Dunod, 2000.
- **Introduction à la programmation objet en Java**, J. Brondeau, Dunod, 1999.
- **Le langage Java – Programmer par l'exemple**, T. Leduc, D. Leduc, Technip, 2000.
- **Algorithmique et programmation objet en Java**, V. Granet, Dunod, 2001.

Bibliographie Java (2/2)

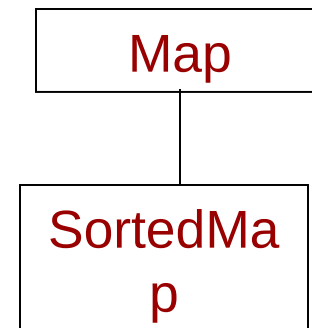
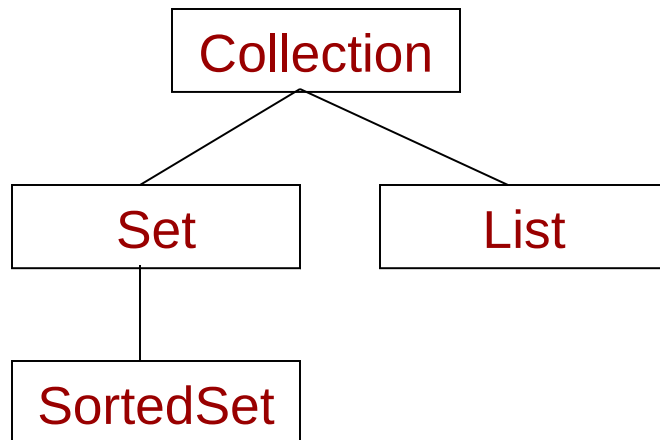
- **Initiation à l'algorithmique objet**, A. Cardon, C. Dabancourt, Eyrolles, 2001.
- **Thinking in Java**, B. Eckel, Prentice Hall, 2000.
Disponible sous forme électronique à l'URL
<http://www.BruceEckel.com>
- **Le poly de Java**, H. Garetta, Polycopié, Université de la Méditerranée, 2003.
- Site Java de Sun : <http://java.sun.com>



COLLECTIONS

Collections Java

- Java fournit un ensemble d'interfaces et de classes facilitant la manipulation de collections d'objets.
- Une collection est un objet qui sert à stocker d'autres objets.
- Interfaces sur les collections organisées en deux catégories : **Collection** et **Map**.



Collections Java : interfaces

- **Collection** : groupe d'objets où la duplication peut être autorisée.

L'interface Collection spécifie des méthodes comme **add**, **remove**, **addAll**, **removeAll**, **contains**, **containsAll**, **size**, etc.

- **Set** : objets collections non ordonnées et sans doublons. **SortedSet** est un Set trié.
- **List** : collections ordonnées autorisant des doublons. Chaque objet possède une position dans la séquence et l'interface offre des méthodes permettant l'accès direct à un objet donné.
- **Map** : tableaux associatifs (dictionnaires) permettant de retrouver une information grâce à un index. **SortedMap** est un Map trié.

Collections Java : implémentations

Classes d'implémentation

Table de
Hachage

Tableau
(Vecteur)

Arbre

Liste
chaînée

Set

HashSet

TreeSet

Interfaces

List

ArrayList

LinkedList

Map

HashMap

TreeMap

Collections Java : Itérateurs

- Les collections peuvent être parcourues à l'aide d'itérateurs : interface **Iterator**.

- Exemple :

```
import java.util.*;
public test_iterateur {
    public static void main (String[] args) {
        Set ufrs = new TreeSet();
        ufrs.add("sat"); ufrs.add("seg"); ufrs.add("sjp");
        // iterer
        Iterator iter = ufrs.iterator();
        while (iter.hasNext()) {
            String ufr = (String) iter.next();
            System.out.println(ufr);
        }
    }
}
```

Collections Java : Map

- Les objets Map permettent d'établir une correspondance entre deux classes d'objets (par exemple des termes anglais et français).
- Ensemble de paires (clé, valeur) tq l'ensemble des clés est sans doublons.
- L'interface interne **Entry** permet de manipuler les éléments d'une paire au moyen des méthodes :
 - ✓ **getKey** et **getValue** : retournent respectivement la clé et la valeur associée à cette clé.
 - ✓ **setValue** : permet de modifier la valeur d'une paire.

Collections Java : Map

L'interface Map offre des méthodes permettant d'itérer sur les clés, les valeurs et sur les paires:

```
Map m = new HashMap();  
// sur les clés  
for (Iterator i = m.keySet().iterator(); i.hasNext();) {  
    System.out.println(i.next());  
}  
// sur les valeurs  
for (Iterator i = m.values().iterator(); i.hasNext();) {  
    System.out.println(i.next());  
}  
// sur les paires clé/valeur  
for (Iterator i = m.entrySet().iterator(); i.hasNext();) {  
    Map.Entry e = (Map.Entry)i.next();  
    System.out.println(e.getKey() + "/" + e.getValue());  
}
```

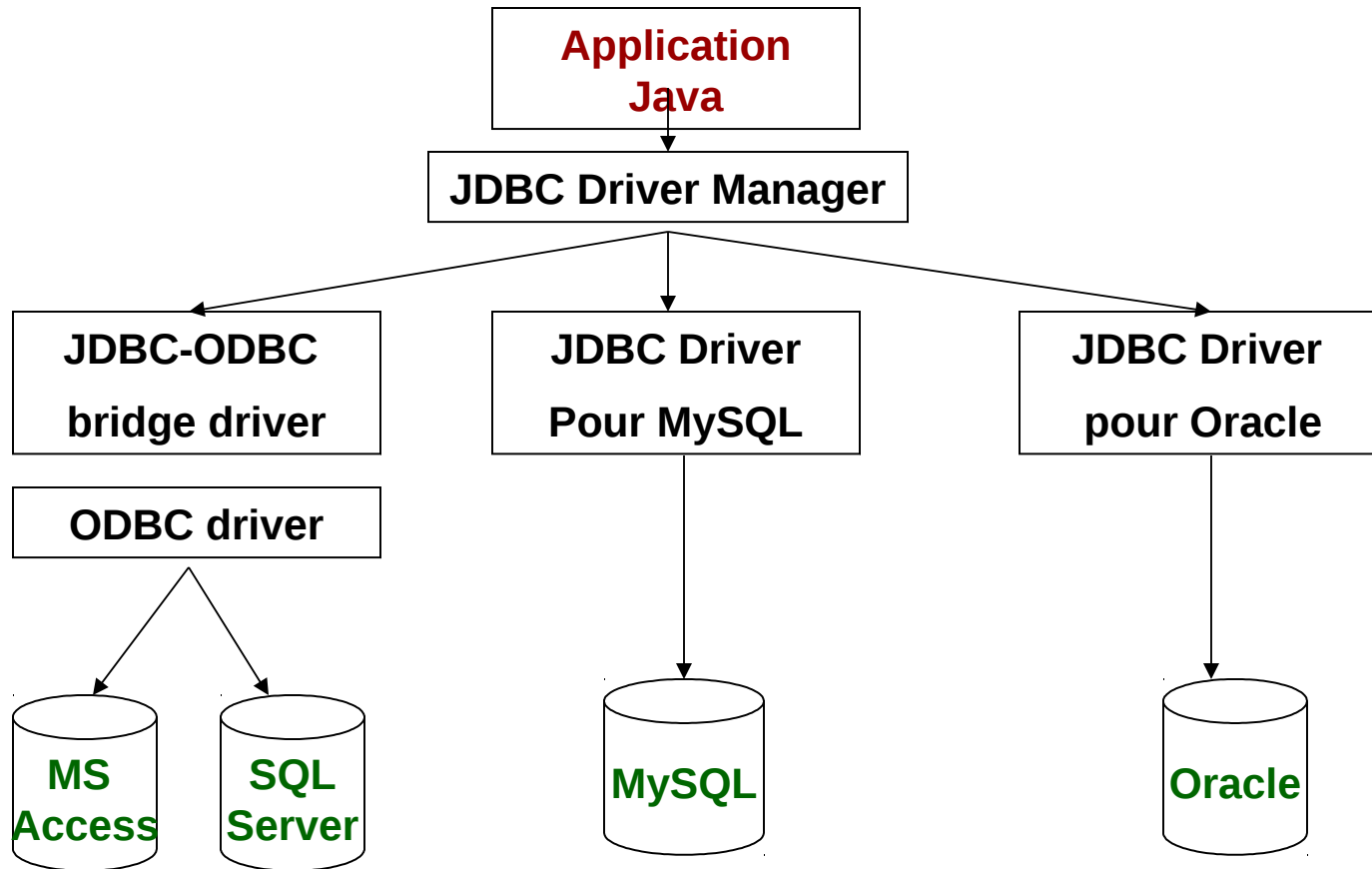


JDBC

JDBC

- *Java Database Connectivity*
- API Java permettant l'accès aux BD (MS Access, SQL Server, MySQL, PostgreSQL, Oracle)
- Fonctionne en client/serveur (appl. Java / SGBD)
- Les classes de l'API se trouvent dans **java.sql**

JDBC : architecture



JDBC : architecture

- **JDBC Driver Manager** : Gestionnaire de drivers permettant à chaque application de charger le(s) driver(s) dont il a besoin.
- **Driver JDBC** : gère les détails de communication avec un type de SGBD.
 - ✓ Un driver par SGBD (Oracle, MySQL, ...)
 - ✓ JDBC-ODBC : driver générique pour toutes les sources accessibles via ODBC (*Open DataBase Connectivity*. interface Microsoft permettant la communication entre des clients bases de données fonctionnant sous Windows et les SGBD du marché).

JDBC : fonctionnement

JDBC fonctionne comme suit :

- Création d'une connexion à la BD
- Envoi de requêtes SQL (pour récupérer ou maj des données)
- Exploitation des résultats provenant de la base
- Fermeture de la connexion

JDBC : exemple de connexion

Connexion à une base MySQL nommée MABD

```
import java.sql.*;
public test_jdbc {
    public static Connection initConnection () {
        Connection co = null;
        String url = "jdbc:mysql://localhost/MABD";
        try{
            Class.forName("org.gjt.mm.mysql.Driver");
            co = DriverManager.getConnection(url);
        }
        catch (SQLException se) {
            System.out.println("connexion impossible");
        }
        return co;
    }
    ...
}
```

JDBC : Requêtes SQL

- JDBC permet divers types de requêtes SQL : interrogation, maj, création de tables.
- Les objets suivants sont disponibles :
 - ✓ **ResultSet** : contient des informations sur une table (noms des colonnes) ou le résultat d'une requête SQL.
 - ✓ `ResultSet rs = (ResultSet)st.executeQuery("Select ...");`
 - ✓ **ResultSetMetaData** : contient des informations sur le nom et le type des colonnes d'une table
 - ✓ `ResultSetMetaData rsmd = rs.getMetaData();`
 - ✓ `int nbre_Colonne = rsmd.getColumnCount();`
 - ✓ **DataBaseMetaData** : contient les informations sur la BD (noms des tables, index, etc.)

JDBC : Requêtes SQL

- Pour envoyer une requête SQL “normale”, il faut initialement créer un objet **Statement** à partir de l'objet Connection:
`Statement st = co.createStatement();`
- **PreparedStatement** correspond à une requête SQL précompilée et gérée par le client JDBC
- **CallableStatement** correspond à une requête SQL stockée, écrite dans le langage de la BD et gérée par celle-ci.
- Ces deux types de requêtes peuvent être paramétrées et exécutées plusieurs fois.

JDBC : exemple de Requête SQL

```
public test_jdbc {  
    public static void main (String[] args) {  
        Connection maCo = initConnection();  
        if (maCo == null) return;  
        String req = "Select nom, sexe,age from  
Personne";  
        try{  
            Statement st = maCo.createStatement();  
            ResultSet rs = st.executeQuery(req);  
            while (rs.next()) {  
  
                System.out.print("nom :"+rs.getString(1));  
  
                System.out.print("sexe:"+rs.getString("sexe"));  
  
                System.out.println("age :"+rs.getDouble(3));  
            }  
            rs.close(); st.close(); maCo.close();  
        }  
        catch (SQLException se) {  
            System.out.println("connexion  
impossible");  
        }  
    }  
}
```

Fin du Chapitre 5.